Monitoring Applications and Infrastructure

SPL-TF-300-SYMON3-1 - Version 1.0.3

Note: Do not include any personal, identifying, or confidential information into the lab environment. Information entered may be visible to others.

Corrections, feedback, or other questions? Contact us at *AWS Training and Certification*.

Lab overview

As the CloudOps engineer for your company, you are asked to create a monitoring solution for one of the company's more heavily used application servers. The monitoring needs to achieve situational awareness to provide timely and effective responses. To achieve this task you: configure the telemetry agent on your Amazon Elastic Compute Cloud (Amazon EC2) instances, create a CloudWatch dashboard to display and view only the metrics you want, configure an automated notification system for events that go past a defined alarm threshold, and manually test the created alarms using the AWS Command Line Interface (AWS CLI).

You practice setting up and monitoring metrics for business application events, defining relevant event thresholds for metrics, creating an automated notification, and remediation when metric thresholds are exceeded. Tasks include making use of a locally installed Amazon CloudWatch agent for collecting and monitoring telemetry from Amazon EC2 instances. The published metrics are used with customized dashboards, alarming, and notification services.

Monitoring of applications and infrastructure is much more than tracking latency and CPU usage of your systems. Relevant metric creation, log collection, event monitoring and alerting, and remediation are all integrated parts of a mature and robust monitoring system. Monitoring provides tools for making informed business decisions on capacity planning, costs, utilization, and security.

Objectives

By the end of this lab, you should be able to do the following:

- Configure telemetry on Amazon EC2 instances using the Amazon CloudWatch agent, AWS Sessions Manager, command documents, and Parameter Store.
- Create a CloudWatch dashboard to display CloudWatch agent metrics.
- Subscribe to Amazon Simple Notification Service (Amazon SNS) topics for automatic notifications.
- Configure CloudWatch alarms for monitoring and notification when specific metric thresholds are crossed.
- Manually test any CloudWatch alarm in your AWS environment using the AWS CLI.
- Create an AWS Lambda based Canary alarm for a web server.

Technical knowledge prerequisites

This lab requires the following prerequisites:

- Access to a computer with internet and Microsoft Windows, macOS X, or Linux.

- An internet browser, such as Google Chrome, Mozilla Firefox, or Microsoft Internet Explorer 11 (previous versions of Internet Explorer are not supported).
- Basic knowledge of AWS Systems Manager, Amazon CloudWatch, Amazon SNS, and AWS Lambda.

Icon key

Various icons are used throughout this lab to call attention to different types of instructions and notes. The following list explains the purpose for each icon:

- **Caution:** Information of special interest or importance (not so important to cause problems with the equipment or data if you miss it, but it could result in the need to repeat certain steps).
- **Command:** A command that you must run.
- **Copy edit:** A time when copying a command, script, or other text to a text editor (to edit specific variables within it) might be easier than editing directly in the command line or terminal.
- **Expected output:** A sample output that you can use to verify the output of a command or edited file.
- **File contents:** A code block that displays the contents of a script or file you need to run that has been pre-created for you.
- **Hint:** A hint to a question or challenge.
- **Knowledge check:** An opportunity to check your knowledge and test what you have learned.
- **Learn more:** Where to find more information.
- **Note:** A hint, tip, or important guidance.
- **Refresh:** A time when you might need to refresh a web browser page or list to show new information.
- **Task complete:** A conclusion or summary point in the lab.
- **Warning:** An action that is irreversible and could potentially impact the failure of a command or process (including warnings about configurations that cannot be changed after they are made).

Lab environment

The following diagram represents the major components used in this lab. The numbers represent the logical workflow of the architecture in this lab.

The architecture diagram of the lab environment.

Image description: The preceding diagram depicts the use of AWS Systems Manager to make operational changes to an Amazon EC2 instance. You use the Amazon CloudWatch service to create customized dashboards for specified metrics. You also use the Amazon CloudWatch service to initiate alarms when specified metrics pass defined thresholds. You pair CloudWatch alarms with a topic from Amazon SNS to push notifications out to your subscribed email. You verify the alarming and notification mechanisms you built in the lab are working by using the Amazon CLI. Finally, you create and test an AWS Lambda function which has its own functional CloudWatch alarm.

AWS services not used in this lab

AWS service capabilities used in this lab are limited to what the lab requires. Expect errors when accessing other services or performing actions beyond those provided in this lab guide.

Start lab

1. To launch the lab, at the top of the page, choose **Start Lab**.

   Caution: You must wait for the provisioned AWS services to be ready before you can continue.

2. To open the lab, choose

   Open Console

   You are automatically signed in to the AWS Management Console in a new web browser tab.

   Warning: Do not change the **Region** unless instructed.

Common sign-in errors

Error: Choosing Start Lab has no effect

In some cases, certain pop-up or script blocker web browser extensions might prevent the **Start Lab** button from working as intended. If you experience an issue starting the lab:

- Add the lab domain name to your pop-up or script blocker's allow list or turn it off.
- Refresh the page and try again.

Task 1: Install the CloudWatch agent on the Amazon EC2 instance for monitoring

In this task, you use AWS Systems Manager to install the CloudWatch agent to an Amazon EC2 instance.

You can use the CloudWatch agent to collect both system metrics and log files from Amazon EC2 instances and on-premises servers. The agent supports both Windows Server and Linux Operating Systems and permits you to select the metrics to be collected, including sub-resource metrics such as per-CPU core. Amazon Web Services (AWS) recommends that you use the CloudWatch agent to collect metrics and logs instead of using the monitoring scripts.

Learn more: Refer to *Collect metrics, logs, and traces with the CloudWatch agent* in the **Additional resources** for more information.

The general pre-requisites for installing the CloudWatch agent on an instance using the Systems Manager service are as follows:

- Appropriate AWS Identity and Access Management (IAM) credentials for the CloudWatch and Systems Manager service actions. More details on these roles can be found in the AWS documentation.
- A Systems Manager agent installed on the Amazon EC2 instance or on-premise server to be monitored.
- Routing and network access to the internet or AWS service endpoints, as the service agents make use of ports 443 and 80 for communication with the instance.

Note: For this lab, all the pre-requisite requirements needed to use Sessions Manager and CloudWatch agent with the Amazon EC2 instance are met.

Task 1.1: Update the Systems Manager agent installed on the Amazon EC2 instance

Update the Systems Manager agent on the Amazon EC2 instance by using a Systems Manager command document. Unless you require a specific version of software, updating the installed software and its dependencies to the latest production versions is a best practice.

Note: This lab is built to use Systems Manager and the CloudWatch console for viewing metrics.

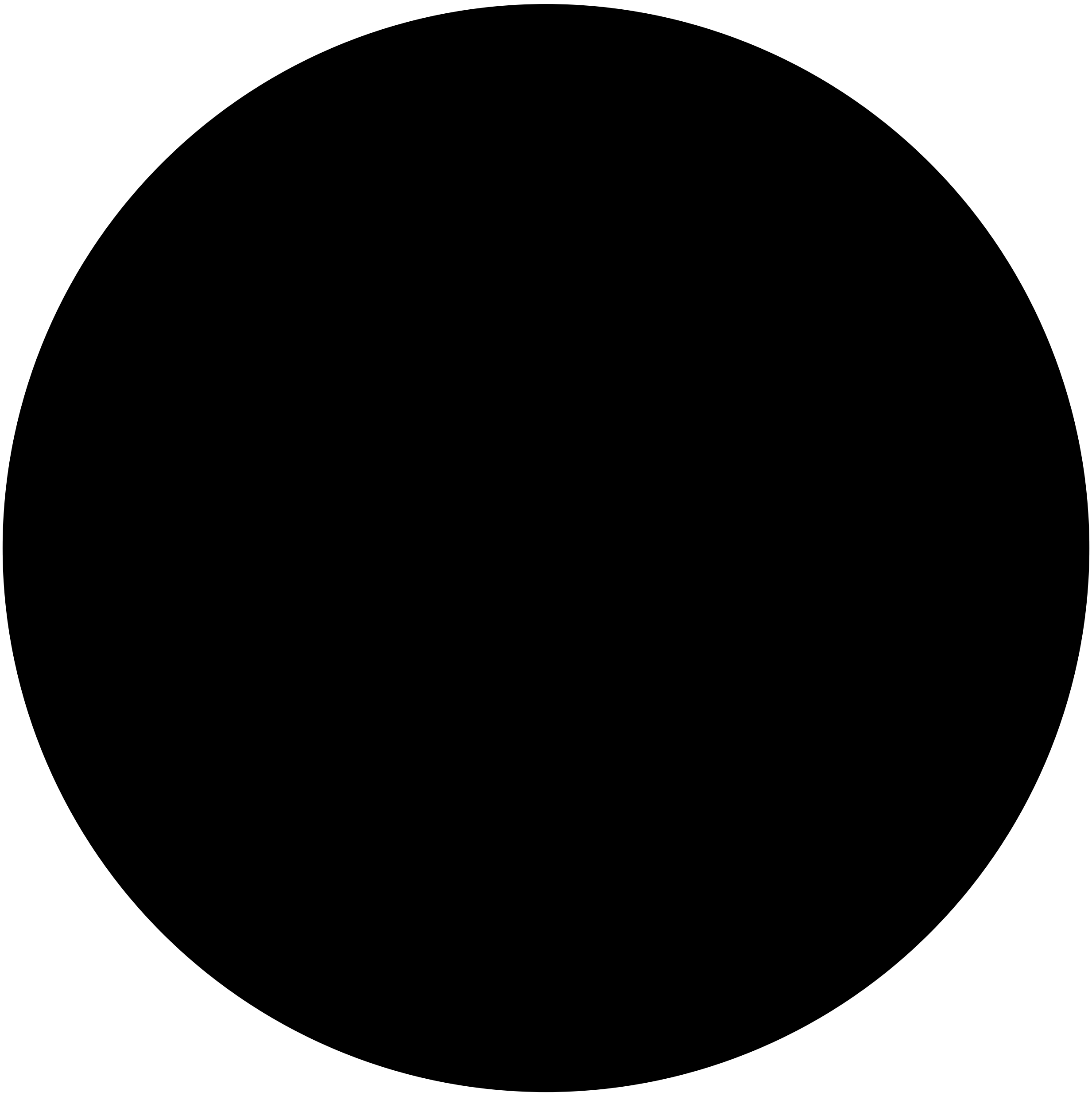
Depending on how your infrastructure was deployed in your AWS account, you may need to install the Systems Manager agent prior to installing the CloudWatch agent. The Systems Manager agent is available for both Linux and Windows Operating Systems.

Learn more: Refer to *Working with SSM Agent on EC2 instances for Linux* in the **Additional resources** section for more information.

The Systems Manager agent comes pre-installed on the Amazon Linux 2023 Operating System, so you do not need to install it, only to update it.

3. At the top of the AWS Management Console, in the search bar, search for and choose Systems Manager
.

Note: If you see the following error banner on top of the page, you can safely ignore it for this lab.



Missing permissions for the new Systems Manager experience

You don’t have some of the permissions required to use the new Systems Manager experience.

4. In the left navigation pane, in the **Change Management Tools** section, choose **Documents**.

Note: The navigation bar is located on the left side of the console. You may need to expand the navigation bar by choosing the menu icon .

5. On the **AWS Systems Manager Documents** page, in the **Owned by Amazon** tab, use the search bar to search for the command document named

   AWS-UpdateSSMAgent

   .

6. Select the link for the **AWS-UpdateSSMAgent** command document from the search results list.

The **Description** page for the command document is displayed.

Note: To review the contents of this, or any Systems Manager command document, select the link for name of the command document and choose the **Content** tab on the subsequent details page.

7. Choose **Run command**.

On the **Run a command** page, configure the following:

8. In the **Target selection** section, select the radio button next to **Choose instances manually**.
9. In the **Instances** section, select the checkbox next to the Amazon EC2 instance named **AppServer**.
10. Expand the **Output options** section if not expanded already, and deselect **Enable an S3 bucket**.
11. Choose **Run**.

The **Run Command summary** page is displayed.

12. **Refresh:** Choose the refresh button in the console to periodically refresh the output on the page until the value of **Overall status** in the **Command status** section displays Success.
13. In the **Targets and outputs** section, choose the link for the **Instance ID**.
14. Expand the **Output** sections of all the **Steps**.
15. Review the command outputs and discover which version of the Systems Manager agent was installed on the Amazon EC2 instance.

Hint: Step 1 of this command document determines the operating system deployed, and step 2 will have output related to the agent update.

Task complete: You have used a Systems Manager command document to update the Systems Manager agent installed on the Amazon EC2 instance and you verified which version is installed.

Task 1.2: Use AWS Systems Manager to install the CloudWatch agent to a managed Amazon EC2 instance

Use a Systems Manager command document to install the CloudWatch agent onto a managed Amazon EC2 instance.

16. In the left navigation pane, in the **Change Management Tools** section, choose **Documents**.
17. On the **AWS Systems Manager Documents** page, in the **Owned by Amazon** tab, use the search bar to search for the command document named

    AWS-ConfigureAWSPackage

    .
18. Select the link for the **AWS-ConfigureAWSPackage** command document from the search results list.

The **Description** page for the command document is displayed.

19. Choose **Run command**.

On the **Run a command** page, configure the following:

20. In the **Command parameters** section, for **Name**, enter

    AmazonCloudWatchAgent
    .

21. In the **Target selection** section, select the radio button next to **Choose instances manually**.

22. In the **Instances** section, select the checkbox next to the Amazon EC2 instance named **AppServer**.

23. Expand the **Output options** section if not expanded already, and deselect **Enable an S3 bucket**.

24. Choose **Run**.

The **Run Command summary** page is displayed.

25. **Refresh:** Choose the refresh button in the console to periodically refresh the output on the page until the value of **Overall status** in the **Command status** section displays Success.

Task complete: You have used a Systems Manager command document to install the AWS CloudWatch agent to a specific Amazon EC2 instance.

Task 2: Configure and start the CloudWatch agent installed on an Amazon EC2 instance

Utilize Systems Manager command documents to configure and start a CloudWatch agent installed on an Amazon EC2 instance.

With the CloudWatch agent installed to the instance it is now time to configure and start it. Because the lab architecture is using only an Amazon EC2 instance and no proxy servers are involved, the default settings found in the file located on the Amazon EC2 instance at

/opt/aws/amazon-cloudwatch-agent/etc/common-config.toml
are fine as is. This is the file you can configure if your environment uses on-premises or remote proxy servers.

Note: You must create a CloudWatch agent configuration file before starting the CloudWatch agent on any server .

The CloudWatch agent configuration file is a JSON file that specifies the metrics and logs that the agent is to collect, including any custom metrics. A wizard included with the CloudWatch Agent creates an initial configuration file which you can manually edit or you can create the configuration file yourself from scratch.

Command: That wizard can be started on Amazon Linux 2023 OS with the following command:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
```

If you create or modify the configuration file manually the process is more complex, but you have more control over the metrics collected and can specify additional metrics not available through the wizard.

Any time you change the CloudWatch agent configuration file the CloudWatch agent service must be restarted before the changes take effect.

After you have created a CloudWatch agent configuration file you can save it locally as a JSON file and use it when installing the agent on your other servers. Alternatively, you can store the configuration file in Systems Manager Parameter Store for later use with command documents. This is the approach you use in this lab.

For this lab, the configuration file is already provided to you and saved in the Parameter Store.

Task 2.1: View the CloudWatch agent configuration file

Review the config file that has been provided for the lab and stored in Parameter Store.

You can store the configuration file locally and reference it when starting the CloudWatch agent service. However, when you store the configuration files in Parameter Store and use it in conjunction with Systems Manager command documents, you can conveniently deploy the same configuration of the CloudWatch agent on many different servers. The process can be used for automation. When you store the configuration file on Parameter Store, it provides an audit trail, supports AWS tagging, and the configuration document can be versioned.

26. In the left navigation pane, in the **Application Tools** section, choose **Parameter Store**.

27. Choose the link for the parameter called **AgentConfigFile** from the console.

28. Review the raw JSON that makes up the configuration file under the **Value** section of the document.

Task complete: You have located your CloudWatch agent configuration file in Parameter Store.

Task 2.2: Start the CloudWatch agent

Start the CloudWatch agent on an Amazon EC2 instance where it has been installed and configured, and verify that the CloudWatch agent is running.

29. Expand the **Systems Manager** navigation bar if necessary by choosing the menu icon on the left side of the AWS console.

30. In the left navigation pane, in the **Change Management Tools** section, choose **Documents**.

31. On the **AWS Systems Manager Documents** page, in the **Owned by Amazon** tab, use the search bar to search for the command document named

    AmazonCloudWatch-ManageAgent

    .

32. Select the link for the **AmazonCloudWatch-ManageAgent** command document from the search results list.

The **Description** page for the command document is displayed.

33. Choose the **Content** tab and examine the document.

34. Notice that the document contains instructions for more than one type of Operating System.

When this command document is ran, you should expect a step error when the OS specified does not match the OS of Amazon EC2 instance. Why is this expected? Do you think the command document will succeed overall or not? Why?

35. Choose **Run command**.

On the **Run a command** page, configure the following:

36. In the **Command parameters** section:

- Ensure **configure** is selected from the **Action** drop-down menu.
- Ensure **ssm** is selected from the **Optional Configuration Source** drop-down menu.
- Enter
  AgentConfigFile
  in the **Optional Configuration Location** box. This is the name of the document located in Parameter Store.
- Ensure **yes** is selected from the **Optional Restart** drop-down menu.

37. In the **Target selection** section, select the radio button next to **Choose instances manually**.

38. In the **Instances** section, select the checkbox next to the Amazon EC2 instance named **AppServer**.

39. Expand the **Output options** section if not expanded already, and deselect **Enable an S3 bucket**.

40. Choose **Run**.

The **Run command** summary page is displayed.

41. **Refresh:** Choose the refresh button in the console to periodically refresh the output on the page until the value of **Overall status** in the **Command status** section displays Success.

42. In the **Targets and outputs** section, choose the link for the **Instance ID**.

The **Output** page is displayed.

43. Expand and examine **Step 1 - Output** and **Step 2 - Output** and **Step 3 - Output** sections.

44. Locate the output section that is relevant to the OS found on the instance.

The first line in the log shows what command was used on the instance. Can you verify that the config file was successfully fetched from Parameter Store?

Knowledge check: Were any errors present? If so what were they and do they make sense?

The last line in the log will show the service is requested to restart after being configured.

Task complete: You have used a command document to configure and start the CloudWatch agent on an Amazon EC2 instance.

Task 2.3: Use SSM documents to verify the CloudWatch agent status on an Amazon EC2 instance

Run the command document name **AmazonCloudWatch-ManageAgent** to query the status of the agent on an Amazon EC2 instance.

Command: The status of an agent running on a Linux server could be verified by running the following command:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-m ec2 \
-a status
```

Command: Or on a server running Windows Server, enter the following in PowerShell as an administrator:

```
& $Env:ProgramFiles\Amazon\AmazonCloudWatchAgent\amazon-cloudwatch-agent-ctl.ps1 -m ec2 -a
```

This command line process is too manual for managing a large fleet. For this reason, you use the command document to query the status of a service installed on an Amazon EC2 instance.

Any command document process can be applied to any number of instances or on-premises servers which are in your fleet managed by AWS Systems Manager.

45. Expand the **Systems Manager** navigation bar if necessary by choosing the menu icon on the left side of the AWS console.

46. In the left navigation pane, in the **Node Tools** section, choose **Run Command**.

47. Choose **Run command**.

48. On the **Run a command** page, in the **Command document** section, use the search bar to search for the command document named

    AmazonCloudWatch-ManageAgent

    .

49. Select the **AmazonCloudWatch-ManageAgent** link from the search results list.

The **Description** page for the command document is displayed.

50. Choose **Run command**.

On the **Run a command** page, configure the following:

51. In the **Command parameters** section:

- Choose **status** from the **Action** drop-down menu.
- Choose **default** from the **Optional Configuration Source** drop-down menu.
- Leave **Optional Configuration Location** blank.

52. In the **Target selection** section, select the radio button next to **Choose instances manually**.

53. In the **Instances** section, select the checkbox next to the Amazon EC2 instance named **AppServer**.

54. Expand the **Output options** section if not expanded already, and deselect **Enable an S3 bucket**.

55. Choose **Run**.

The **Run command** summary page is displayed.

56. **Refresh:** Choose the refresh button in the console to periodically refresh the output on the page until the value of **Overall status** in the **Command status** section displays Success.

57. In the **Targets and outputs** section, choose the link for the **Instance ID**.

The **Output** page is displayed.

58. Expand and examine **Step 1 - Output**, **Step 2 - Output** and **Step 3 - Output** sections.

59. Locate the Output section that is relevant to the OS found on the instance.

Expected output: If the agent is running, the output contains lines that are similar to the following:

```
************************
**** EXAMPLE OUTPUT ****
************************
{
"status": "running",
"starttime": "2023-11-29T21:05:44+0000",
"configstatus": "configured",
"version": "1.300031.0b313"
}
```

Task complete: You have used a command document to verify the status of an agent running on an Amazon EC2 service.

Task 2.4: Verify CloudWatch agent metrics have been pushed to the CloudWatch service

Verify that the CloudWatch agent metrics are appearing on the CloudWatch dashboard.

60. At the top of the AWS Management Console, in the search bar, search for and choose

    CloudWatch
    .

61. In the left navigation pane, in the **Metrics** section, choose **Classic metrics**.

62. From the **Custom namespaces** section, locate the namespace exactly named **CWAgent**.

Refresh: If you do not find the **CWAgent** metric namespace, wait for as long as 5 minutes before refreshing the page.

Task complete: If the metric namespace named *CWAgent* is on the CloudWatch metrics console, you have verified that the CloudWatch agent is correctly pushing metrics from the Amazon EC2 instance to the CloudWatch service.

Task 3: Create a CloudWatch dashboard

In this task, you create a customized CloudWatch dashboard consisting of metrics from both the installed CloudWatch agent as well as standard CloudWatch metrics. Dashboards are a great way to setup a tailored view of only the metrics you need for operations, without having to sort through all metrics available.

63. In the left navigation pane, choose **Dashboards**.

64. Choose **Create dashboard**.

The **Create new dashboard** pop-up window is displayed.

65. For the **Dashboard name**,enter

    myDashboard
    .

66. Choose **Create dashboard**.

The new CloudWatch dashboard is created, and the **Add widget** pop-up window is displayed.

67. Under **Widget type**, select the radio button next to the **Line** widget type.

68. Choose **Next**.

The **Add metric graph** pop-up window is displayed.

69. From the **Custom namespaces** section, select the namespace exactly named **CWAgent**.

70. Select the metric namespace that is exactly named **ImageId, InstanceId, InstanceType**.

71. Select the checkbox in the row where the **Metric Name** column has a metric named **mem_used_percent** for instance named **AppServer**.

72. Choose **Create widget**.

The CloudWatch dashboard **myDashboard** now displays a singular graph for the percentage of memory used on your Amazon EC2 instance.

73. Choose **Save**.

Add a second widget to the CloudWatch dashboard **myDashboard**.

74. Choose the **+** button to add second widget.

The **Add widget** pop-up window is displayed.

75. Under **Widget type**, select the radio button next to the **Stacked area** widget type.

76. Choose **Next**.

The **Add metric graph** pop-up window is displayed.

77. Choose the edit icon which is next to the words, **mem_used_percent**.

78. Enter

    AppServer network activity

    .

79. Choose **Apply**.

The metric graph now has a name.

80. From the **AWS namespaces** section, choose the namespace exactly named **EC2**.

81. Choose the metric namespace exactly named **Per-Instance Metrics**.

82. Select the checkbox in the row where the **Metric Name** column has a metric named **NetworkIn** for the instance named **AppServer**.

83. Select the checkbox in the row where the **Metric Name** column has a metric named **NetworkOut** for the instance named **AppServer**.

84. Choose **Create widget**.

The CloudWatch dashboard **myDashboard** now displays an additional graph for the amount of network activity on your Amazon EC2 instance.

85. Choose **Save**.

Learn more: Refer to *CloudWatch metrics that are available for your instances* in the **Additional resources** section for more information.

Task complete: You created a CloudWatch dashboard consisting of only a few specific metrics and graphs.

Task 4: Subscribe to an AWS Simple Notification Service topic

In this task, you subscribe your email address to a provided Amazon Simple Notification Service (Amazon SNS) topic named **SysOpsTeamPager**. The Amazon SNS topic is used to alert all subscribed engineers when a specified CloudWatch alarm is triggered.

86. At the top of the AWS Management Console, in the search bar, search for and choose

    Simple Notification Service

    .

87. From the left navigation menu, choose **Topics**.

88. Choose the link for **SysOpsTeamPager** topic from the list.

89. Choose **Create subscription**.

90. On the **Create subscription** page:

- For **Protocol**, use the drop-down menu and select **Email**.
- For **Endpoint**, enter a valid email address you can access.

Note: In your personal AWS environment, this might be an alias for all of the CloudOps engineers or it might be an individual subscriber. Individual subscribers receive an email and have to confirm the subscription prior to receiving future notifications from the topic.

91. Choose **Create subscription**.

A banner message with text similar to, *Subscription to SysOpsTeamPager created successfully.* is displayed at the top of the page letting you know the email address was successfully registered to the Amazon SNS topic.

92. Open the inbox of the email address you entered for the subscription.

93. Locate a recent message from **SysOpsTeamPager** **no-reply@sns.amazonaws.com**.

Note: It may take up to 5 minutes to receive the email, depending on your email server.

94. Choose the **Confirm subscription** link contained in the email.

A page is opened confirming the subscription.

95. Close the Amazon SNS topic subscription confirmation page.

Task complete: You created a new Amazon SNS topic and successfully subscribed to the topic, permitting Amazon SNS to push new messages from this topic to your email address.

Task 5: Create specific CloudWatch alarms for metrics

In this task, you create an alarm to be triggered when a metric passes a defined threshold. CloudWatch alarms can be made for any of the CloudWatch metric. You are not limited to metrics you added to your customized CloudWatch dashboard for monitoring in a previous task.

96. At the top of the AWS Management Console, in the search bar, search for and choose

    CloudWatch
    .

97. From the left navigation menu, choose **Alarms**.

98. Choose **Create alarm**.

This begins the CloudWatch Alarms guided setup process.

99. On the **Specify metric and conditions** page, for **Metric**, choose **Select metric**.

A **Select metric** pop-up window is displayed. This is similar to the process for selecting metrics for monitoring purposes on a CloudWatch dashboard, but now you are selecting metrics for alarming purposes.

100. From the **AWS namespaces** section, choose the namespace exactly named **EC2**.

101. Choose the metric namespace exactly named **Per-Instance Metrics**.

102. Select the check box in the row where the **Metric Name** column has a metric named **StatusCheckFailed_System** for the instance named **AppServer**.

103. Choose **Select metric**.

On the **Specify metric and conditions** page, configure the following:

104. On the **Conditions** section:

- For **Threshold type**, choose **Static**.
- For **Whenever StatusCheckFailed_System is…**, choose **Greater/Equal >= threshold**.
- For **than…**, enter
  1
  .

105. Choose **Next**.

On the **Configure actions** page, configure the following:

106. In the **Notification** section:

- For **Send a notification to the following SNS topic**, ensure **Select an existing SNS topic** is selected.
- For **Send a notification to…**, use the search bar and select **SysOpsTeamPager**.

107. Choose **Next**.

On the **Add alarm details** page, configure the following:

108. For **Alarm name**, enter

    AppServerSystemsCheckAlarm

    .

109. **Copy edit:** For **Alarm description - *optional***, enter the following:

```
This alarm is triggered when an Amazon EC2 instance fails System Reachability check. It is
```

It is best practice for alarm names to be self-descriptive, and for the alarm descriptions to contain useful information. Only 1024 characters are allowed in alarm descriptions. Be succinct, describe the alarm, link to the Operation team's runbooks, and suggest remediation action.

110. Choose **Next**.

111. On the **Preview and create** page, choose **Create alarm**.

Task complete: You have used the CloudWatch console to create an Alarm for a defined metric and threshold. You also configured the alarm to send notifications to an existing SNS topic. More than one alarm can make use of a single Amazon SNS topic.

Task 6: Test the CloudWatch alarm from the AWS CLI

In this task, you use Sessions Manager to log into an Amazon EC2 instance and use the AWS CLI to temporarily change the state for an alarm. It is always useful to test any alarms you build before you actually need them. When building alarms for any metric, instead of building a unique testing workload for each alarm and waiting for the alarm to trigger, it is quicker to test the alarm and any subsequently triggered actions from other services such as Amazon SNS, AWS Lambda, and Amazon EC2 by activating the alarm state from the command line.

Task 6.1: Use Systems Manager and start a session with an Amazon EC2 instance

Use session manager tool found in Systems Manager and start a session with an Amazon EC2 instance.

112. At the top of the AWS Management Console, in the search bar, search for and choose

    Systems Manager

    .

113. In the left navigation pane, in the **Node Tools** section, choose **Session Manager**.

114. Choose **Start session**.

115. Select the radio button next to **AppServer** Amazon EC2 instance in the **Target instances** section.

116. Choose **Next**.

117. On the **Specify session document - *optional*** page, choose **Next**.

118. On the **Review and launch** page, choose **Start session**.

The session is started in a new tab of your browser.

Task complete: You have started a session with an individual Amazon EC2 instance and have access to the terminal.

Task 6.2: Change the state of a CloudWatch alarm

Command: The following example uses the *set-alarm-state* command to temporarily change the state of an Amazon CloudWatch alarm named “myAlarm” and set it to the ALARM state for testing purposes. The '–region ’ option must be included if an AWS region is not configured for the local profile. Use the region that your lab was launched in. Your lab’s region is located to the left of these instructions.

```
*************************
**** EXAMPLE COMMAND ****
*************************

aws cloudwatch set-alarm-state \
--alarm-name "myAlarm" \
--state-value ALARM \
--state-reason "testing purposes" \
--region us-east-1
```

119. **Command:** Enter the following command in the terminal of the AppServer instance:

```
TOKEN=`curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-
aws cloudwatch set-alarm-state \
--alarm-name "AppServerSystemsCheckAlarm" \
--state-value ALARM \
--state-reason "testing purposes" \
--region $(curl -H "X-aws-ec2-metadata-token: $TOKEN" -v http://169.254.169.254/latest/met
```

Caution: If you receive the error *An error occurred (ResourceNotFound) when calling the SetAlarmState operation: Unknown* when running the command, then double-check the command you used for accuracy. It’s likely that either: the incorrect region was specified or a typo occurred in the command.

This command returns to the prompt in the terminal if successful, or one of the command values contains a typo such as the alarm name.

In the inbox of the email account you subscribed to the Amazon SNS topic named **SysOpsTeamPager**, a new email from the Amazon SNS topic should arrive.

Task complete: You have used the AWS CLI to trigger the alarm state for a specific alarm metric.

Task 6.3: Review the Alarm email and CloudWatch dashboard

Review the email generated by Amazon SNS and the related CloudWatch alarm dashboard.

120. Locate and review the CloudWatch alarm email from **SysOpsTeamPager** **no-reply@sns.amazonaws.com**.

The email is similar to the example below and contains a link to the appropriate CloudWatch dashboard. Notice that the reason you stated for the alarm in the command, is present in the first few lines of the message body. The rest of the body message details the alarm.

File contents: For current data on the alarm, you need to review the alarm and metric data from CloudWatch console.

```
You are receiving this email because your Amazon CloudWatch Alarm "AppServerSystemsCheckAl

View this alarm in the AWS Management Console:
https://us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#s=Alarms&alarm=A

Alarm Details:
- Name:                       AppServerSystemsCheckAlarm
- Description:                This alarm is triggered when an Amazon EC2 instance fails Sy
- State Change:               OK -> ALARM
- Reason for State Change:    testing purposes
- Timestamp:                  Thursday 18 June, 2020 19:24:09 UTC
- AWS Account:                123456789
- Alarm Arn:                  arn:aws:cloudwatch:us-east-1:123456789:alarm:AppServerSystem

Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanOrEqualToThreshold 1.0 for

Monitored Metric:
- MetricNamespace:                    AWS/EC2
- MetricName:                         StatusCheckFailed_System
- Dimensions:                         [InstanceId = i-0c7263cc9e278de8f]
- Period:                             60 seconds
- Statistic:                          Average
- Unit:                               not specified
- TreatMissingData:                   missing


State Change Actions:
- OK:
- ALARM: [arn:aws:sns:us-east-1:123456789:myOperationalAlarm]
- INSUFFICIENT_DATA:
```

121. Choose the URL link in the email that follows the statement “View this alarm in the AWS Management Console”.

Your default web browser opens the CloudWatch console on the alarm that was triggered.

Task complete: You have reviewed the alarm notification sent by Amazon SNS and located the appropriate alarm metric within CloudWatch.

Task 6.4: Return to the Amazon EC2 instance and change the state to OK

Because the alarm was triggered by a test, rather than the metric staying above the threshold defined in the alarm, the alarm state for **AppServerSystemsCheckAlarm** returns to the state OK by itself. However, it is useful to know that the same command can be used to change the state to OK, as well as the state ALARM.

122. Open the Session Manager connection to the AppServer. If the session has timed out, restart the session by closing the browser window and following the same steps from a previous task named *Connect to the instance using AWS Session Manager*.

123. **Command:** Enter the following command at the command terminal of the AppServer Amazon EC2 instance:

```
aws cloudwatch set-alarm-state \
--alarm-name "AppServerSystemsCheckAlarm" \
--state-value OK \
--state-reason "testing purposes" \
--region $(curl -s http://169.254.169.254/latest/meta-data/placement/region)
```

Note: You do not receive any email from the SNS topic for this action because you are putting the alarm into the OK state. Only those alarms that in the ALARM state trigger a notification.

The session is no longer needed for this lab and you may close it.

Task complete: You have triggered a specific CloudWatch alarm, reviewed the alarm dashboard, and then turned the alarm off.

Task 7: Configure an AWS Lambda Canary function to use Amazon EventBridge (CloudWatch Events) for a successful website response and test it

In this task, you build and test an AWS Lambda function called *Canary* from the Lambda blueprints. You also configure a CloudWatch alarm for the Lambda Canary function. The Lambda Canary function queries a website for a response and is configured with a pre-defined expected result. The Lambda function triggers a notification when it receives a result differing from the expected value. Finally, you test the CloudWatch alarm and notification by triggering the alarm intentionally.

Task 7.1: Create the AWS Lambda Canary function

Use the Lambda console to create the AWS Lambda Canary function.

Amazon EventBridge (CloudWatch Events) emits an event every minute, based on the schedule expression. The event triggers the Lambda function, which verifies that the expected string appears in the specified page.

Learn more: Refer to *Invoke a Lambda function on a schedule* in the **Additional resources** section for more information.

124. At the top of the AWS Management Console, in the search bar, search for and choose

     Lambda
     .

125. Choose **Create function**.

On the **Create function** page, configure the following:

126. For **Choose one of the following options to create your function**, choose **Use a blueprint**.

In the **Basic information** section, configure the following:

127. For **Blueprint name**, use the search bar to search for

     canary
     and then select **Schedule a periodic check of any URL**.

128. For **Function name**, enter

     lambda-canary
     .

129. Under **Execution role**, choose **Use another role**.

130. Use the dropdown menu and choose **Lambda-CanaryRole**.
131. Choose **Save**.

In the **EventBridge (CloudWatch Events) trigger** section, configure the following:

132. For **Rule**, choose **Create a new rule**.
133. For **Rule name**, enter

    CheckWebsiteScheduledEvent

    .

134. For **Rule description**, enter

    CheckWebsiteScheduledEvent trigger

    .

135. For **Schedule expression**, enter

    rate(1 minute)

    .

136. In the **Environment variables** section:

- For the Key **site**, enter the value
  https://docs.aws.amazon.com/lambda/latest/dg/welcome.html
  .
- For the Key **expected**, enter the value
  What is AWS Lambda?
  .

137. Choose **Create function**.

The Lambda function has been created and the browser is redirected to the function detail page.

A banner message is displayed with the text: *Congratulations! Your Lambda function "lambda-canary" has been successfully created and configured with CheckWebsiteScheduledEvent as a trigger. Choose Test to input a test event and test your function.*

Task complete: You have created the Lambda Canary function.

Task 7.2: Test the Lambda function

Test the Lambda function with a sample event provided by the Lambda console.

138. Choose the **Test** tab in the middle of the page next to the **Code** tab.
139. For **Event name**, enter

    CanaryTestEvent

    .

140. Choose **Save**.
141. Choose **Test**.

The output from the test is shown in a banner.

142. Choose **Details** from the *Executing function: succeeded* banner to reveal the logs from the test event.

Task complete: You have created a test for the Lambda function.

Task 7.3: Configure a CloudWatch alarm for the Lambda Canary function

Configure an alarm in Amazon CloudWatch that monitors the Canary Lambda function and sends a notification to an Amazon SNS topic when the defined threshold is passed.

143. At the top of the AWS Management Console, in the search bar, search for and choose

    CloudWatch

    .

144. From the left navigation menu, choose **Alarms**.

145. Choose **Create alarm**.

This begins the CloudWatch Alarms guided setup process.

146. On the **Specify metric and conditions** page, for **Metric**, choose **Select metric**.

Create an alarm with the following settings:

147. From the **AWS namespaces** section, choose the namespace exactly named **Lambda**.

148. Choose the metric namespace exactly named **By Function Name**.

149. Select the checkbox in the row where the **Metric name** column has a value **Errors**, and the **FunctionName** column has a value **lambda-canary**.

Note: It can take up to 5 minutes time from creating a Lambda function, until the metrics for that first appear in CloudWatch. If you do not find the lambda-canary metrics, wait for as long as 5 minutes before refreshing the page.

150. Choose **Select metric**.

On the **Specify metric and conditions** page, configure the following:

151. In the **Metric** section:

- For **Statistic**, use the dropdown menu and select **Sum**.
- For **Period**, use the dropdown menu and select **1 minute**.

152. In the **Conditions** section:

- For **Whenever Errors is…**, choose **Greater/Equal >= threshold**.
- For **than…**, enter

  1

  .

153. Choose **Next**.

On the **Configure actions** page, configure the following:

154. In the **Notification section**:

- For **Send a notification to the following SNS topic**, ensure **Select an existing SNS topic** is selected.
- For **Send a notification to…**, use the search bar and select **SysOpsTeamPager**.

155. Choose **Next**.

On the **Add alarm details** page, configure the following:

156. For **Alarm name**, enter

    lambda-canary-alarm
    .

157. For **Alarm description - *optional***, enter

    Lambda Canary did not receive the expected response from the website.
    .

158. Choose **Next**.

159. On the **Preview and create** page, choose **Create alarm**.

Task complete: You have created a new alarm based on metrics collected from the Lambda *canary* function.

Task 7.4: Test the alarm

Test the alarm that has been built for the Lambda function. Rather than manually trigger a specific alarm using the AWS CLI, you intentionally cause errors with a misconfigured update to the Lambda function. This action intentionally breaks the Lambda function and puts the related CloudWatch alarm you previously created into an alarm state.

160. At the top of the AWS Management Console, in the search bar, search for and choose

    Lambda
    .

161. Choose the link for the **lambda-canary** function.

162. Choose the **Configuration** tab.

163. Choose **Environment variables** from the left navigation menu.

164. Choose **Edit**.

On the **Edit environment variables** page, configure the following:

165. Locate the **Key** named **expected** and change the **Value** to

    404
    .

166. Choose **Save**.

167. Choose the **Test** tab.

168. With the **CanaryTestEvent** selected in the **Event name** drop-down menu, choose **Test**.

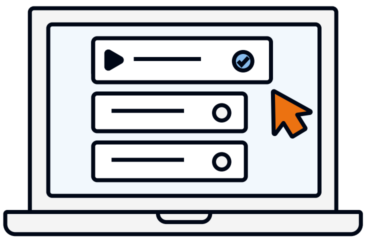
The misconfigured Lambda function generates an error, and the CloudWatch alarm set for the error metric of the Lambda Canary function triggers. The CloudWatch alarm notifies the Amazon SNS topic **SysOpsTeamPager**. This is the topic you subscribed your email address to in a previous task named *Subscribe to the Amazon SNS topic*.

169. **File contents:** Locate and review the CloudWatch alarm email from **SysOpsTeamPager** no-reply@sns.amazonaws.com. Its contents are similar to the following example:

```
You are receiving this email because your Amazon CloudWatch Alarm "lambda-canary-alarm" in

View this alarm in the AWS Management Console:
https://us-west-2.console.aws.amazon.com/cloudwatch/home?region=us-west-2#s=Alarms&alarm=l

Alarm Details:
- Name:                       lambda-canary-alarm
- Description:                lambda-canary-alarm
- State Change:               OK -> ALARM
- Reason for State Change:    Threshold Crossed: 1 out of the last 1 datapoints [1.0 (19/0
- Timestamp:                  Friday 19 June, 2020 18:27:44 UTC
- AWS Account:                123456789
- Alarm Arn:                  arn:aws:cloudwatch:us-west-2:298518306606:alarm:lambda-canar

Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanOrEqualToThreshold 1.0 for

Monitored Metric:
- MetricNamespace:                     AWS/Lambda
- MetricName:                          Errors
- Dimensions:                          [FunctionName = lambda-canary]
- Period:                              300 seconds
- Statistic:                           Sum
- Unit:                                not specified
- TreatMissingData:                    missing


State Change Actions:
- OK:
- ALARM: [arn:aws:sns:us-west-2:123456789:SysOpsTeamPager]
- INSUFFICIENT_DATA:
```

Task complete: You used the Lambda Canary function to simulate the difference between the expected and returned values for a website. This triggered the related CloudWatch alarm and notification services that you setup.

Knowledge Check



This knowledge check is designed to gauge your understanding of the material covered.

Why this matters: Completing this knowledge check will help reinforce your grasp of the content and highlight any areas that may need further review.

Launch knowledge check

Conclusion

You successfully did the following:

- Configured telemetry on Amazon Elastic Compute Cloud (Amazon EC2) instances using the Amazon CloudWatch agent, AWS Sessions Manager, command documents, and Parameter Store.
- Created a CloudWatch dashboard to display CloudWatch agent metrics.
- Subscribed to Amazon Simple Notification Service (Amazon SNS) topics for automatic notifications.
- Configured CloudWatch alarms for monitoring and notification when specific metric thresholds are crossed.
- Manually tested any CloudWatch alarm in your AWS environment using the AWS Command Line Interface (AWS CLI).
- Created an AWS Lambda based Canary alarm for a web server.

End lab

Follow these steps to close the console and end your lab.

170. Return to the **AWS Management Console**.

171. At the upper-right corner of the page, choose **AWSLabsUser**, and then choose **Sign out**.

172. Choose **End Lab** and then confirm that you want to end your lab.

Additional resources

- [Collect metrics, logs, and traces with the CloudWatch agent]
- [Working with SSM Agent on EC2 instances for Linux]
- [CloudWatch metrics available for Amazon EC2 instances]
- [Invoke a Lambda function on a schedule]

For more information about AWS Training and Certification, see *https://aws.amazon.com/training/*.

Your feedback is welcome and appreciated.
If you would like to share any feedback, suggestions, or corrections, please provide the details in our *AWS Training and Certification Contact Form*.

Nice work! Help us make Builder Labs better in just 2 minutes.